Čemu služi glove-wiki-gigaword-50 model?

rad u manjim grupama, pet do deset minuta

Ažurirati kod sa zadnjeg predavanja, zamijeniti GoogleNews-vectors-negative300.bin s glove-wiki-gigaword-50 model

rješenje

```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 36.0 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
import re
import numpy as np

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import classification_report, accuracy_score

import gensim.downloader as api


# ----------------------------
# 1. Učitaj manji skup podataka
# ----------------------------
categories = [
    "rec.sport.baseball",
    "sci.space",
    "talk.politics.mideast",
    "comp.graphics"
]

dataset = fetch_20newsgroups(
    subset="train",
    categories=categories,
    remove=("headers", "footers", "quotes")
)

texts = dataset.data
y = dataset.target
target_names = dataset.target_names

print("Broj dokumenata:", len(texts))
print("Klase:", target_names)


# ----------------------------
# 2. Jednostavna tokenizacija
# ----------------------------
def tokenize(text: str) -> list[str]:
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    return text.split()
```

```
# ----------------------------
# 3. Učitaj pretrained GloVe model
# ----------------------------
# Učitavamo manji pretrained model (GloVe) koji već sadrži vektore riječi.
# Brži je i lakši za rad nego Google News Word2Vec model.

wv = api.load("glove-wiki-gigaword-50")

print("Dimenzija embeddinga:", wv.vector_size)


# ----------------------------
# 4. Dokument -> prosjek word vectors
# ----------------------------
def document_vector(text: str, word_vectors) -> np.ndarray:
    tokens = tokenize(text)

    vectors = []
    for token in tokens:
        if token in word_vectors.key_to_index:
            vectors.append(word_vectors[token])

    if len(vectors) == 0:
        return np.zeros(word_vectors.vector_size, dtype=np.float32)

    return np.mean(vectors, axis=0)


X = np.array([document_vector(text, wv) for text in texts])

print("Shape matrice X:", X.shape)


# ----------------------------
# 5. Train/test split
# ----------------------------
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


# ----------------------------
# 6. Klasifikator
# ----------------------------
clf = LogisticRegression(max_iter=2000)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print()
print(classification_report(y_test, y_pred, target_names=target_names))
```

```
Broj dokumenata: 2338
Klase: ['comp.graphics', 'rec.sport.baseball', 'sci.space', 'talk.politics.mideast']
[==================================================] 100.0% 66.0/66.0MB downloaded
Dimenzija embeddinga: 50
Shape matrice X: (2338, 50)
Accuracy: 0.8589743589743589

                      precision    recall  f1-score   support

        comp.graphics       0.85      0.85      0.85       117
   rec.sport.baseball       0.82      0.94      0.88       119
             sci.space       0.84      0.76      0.80       119
talk.politics.mideast       0.94      0.89      0.91       113

              accuracy                           0.86       468
             macro avg       0.86      0.86      0.86       468
          weighted avg       0.86      0.86      0.86       468
```

Primjena Word2Vec algoritma u klasteriranju podataka

Word2Vec (ili slični embedding modeli) omogućuju nam da tekst pretvorimo u numeričke vektore koji zadržavaju značenje riječi. Kada dokument predstavimo kao prosjek vektora njegovih riječi, možemo primijeniti algoritme poput KMeans kako bismo automatski grupirali tekstove po temama, bez unaprijed definiranih klasa.

koristimo isti dataset kao i prije, ali ovaj put ne dajemo labele

Prvo učitavamo tekstualne podatke iz nekoliko različitih tema. Zatim svaki dokument pretvaramo u jedan vektor tako da prosječimo vektore riječi koje prepoznaje gotovi embedding model. Nakon toga koristimo KMeans da grupiramo dokumente prema sličnosti njihovih vektora, a PCA nam služi samo da te višedimenzionalne podatke prikažemo u dvije dimenzije na grafu.

```
import re
import numpy as np
import matplotlib.pyplot as plt
import gensim.downloader as api

from sklearn.datasets import fetch_20newsgroups
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
```

```
# ------------------------------
# 1. Učitaj tekstualne podatke
# ------------------------------
categories = [
    "rec.sport.baseball",
    "sci.space",
    "talk.politics.mideast",
    "comp.graphics"
```

```
]

dataset = fetch_20newsgroups(
    subset="train",
    categories=categories,
    remove=("headers", "footers", "quotes")
)

texts = dataset.data
true_labels = dataset.target
target_names = dataset.target_names

print("Broj dokumenata:", len(texts))
print("Nazivi stvarnih tema:", target_names)


# ----------------------------
# 2. Jednostavna tokenizacija
# ----------------------------
def tokenize(text: str) -> list[str]:
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    return text.split()
```

```
Broj dokumenata: 2338
Nazivi stvarnih tema: ['comp.graphics', 'rec.sport.baseball', 'sci.space', 'talk.politics.mideast']
```

```
# ----------------------------
# 3. Učitaj gotovi embedding model
# ----------------------------

wv = api.load("glove-wiki-gigaword-50")

print("Dimenzija embeddinga:", wv.vector_size)
```

```
Dimenzija embeddinga: 50
```

```
# ----------------------------
# 4. Dokument -> prosjek word vectors
# ----------------------------
def document_vector(text: str, word_vectors) -> np.ndarray:
    tokens = tokenize(text)

    vectors = []
    for token in tokens:
        if token in word_vectors.key_to_index:
            vectors.append(word_vectors[token])

    if len(vectors) == 0:
        return np.zeros(word_vectors.vector_size, dtype=np.float32)

    return np.mean(vectors, axis=0)


X = np.array([document_vector(text, wv) for text in texts])

print("Shape matrice X:", X.shape)
```

```
Shape matrice X: (2338, 50)
```

```
# ----------------------------
# 5. KMeans klasteriranje
# ----------------------------
k = 4  # znamo da imamo 4 teme
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X)
```

```
print("Broj dodijeljenih klastera:", len(np.unique(cluster_labels)))
```

```
Broj dodijeljenih klastera: 4
```

```
# ----------------------------
# 6. Pogledaj nekoliko dokumenata i njihov klaster
# ----------------------------
for i in range(5):
    print("\n" + "-" * 60)
    print(f"Dokument {i}")
    print(f"Stvarna tema: {target_names[true_labels[i]]}")
    print(f"Dodijeljeni klaster: {cluster_labels[i]}")
    print("Početak teksta:")
    print(texts[i][:250])
```

```
------------------------------------------------------------
Dokument 0
Stvarna tema: comp.graphics
Dodijeljeni klaster: 2
Početak teksta:
Apparently, my editor didn't do what I wanted it to do, so I'll try again.

i'm looking for any programs or code to do simple animation and/or
drawing using fractals in TurboPascal for an IBM
             Thanks in advance

------------------------------------------------------------
Dokument 1
Stvarna tema: rec.sport.baseball
Dodijeljeni klaster: 2
Početak teksta:
[about the infield fly rule]
        Unless he's Deion Sanders, in which case he just heads back to the
dugout and waits for his next base-running-blunder opportunity.
```

```
------------------------------------------------------------
Dokument 2
Stvarna tema: comp.graphics
Dodijeljeni klaster: 2
Početak teksta:
Concerning the proposed newsgroup split, I personally am not in favor of
doing this.  I learn an awful lot about all aspects of graphics by reading
this group, from code to hardware to algorithms.  I just think making 5
different groups out of this i

------------------------------------------------------------
Dokument 3
Stvarna tema: talk.politics.mideast
Dodijeljeni klaster: 3
Početak teksta:

Proven?  Maybe not.  But it can certainly be verified beyond a reasonable doubt.  This
statement and statements like it are a matter of public record.  Before the Six Day War (1967)
I think Nasser and some other Arab leaders were broadcasting these

------------------------------------------------------------
Dokument 4
Stvarna tema: comp.graphics
Dodijeljeni klaster: 0
Početak teksta:
To:All

Hi,

Does anybody have the source code to the externel processes that comes with 3D
Studio, and mabe som kind of DOC for writing the processes your self.


/Lars

+++ Author: Lars_Jorgensen@p7.syntax.bbs.bad.se, Syntax BBS, Denmark
```

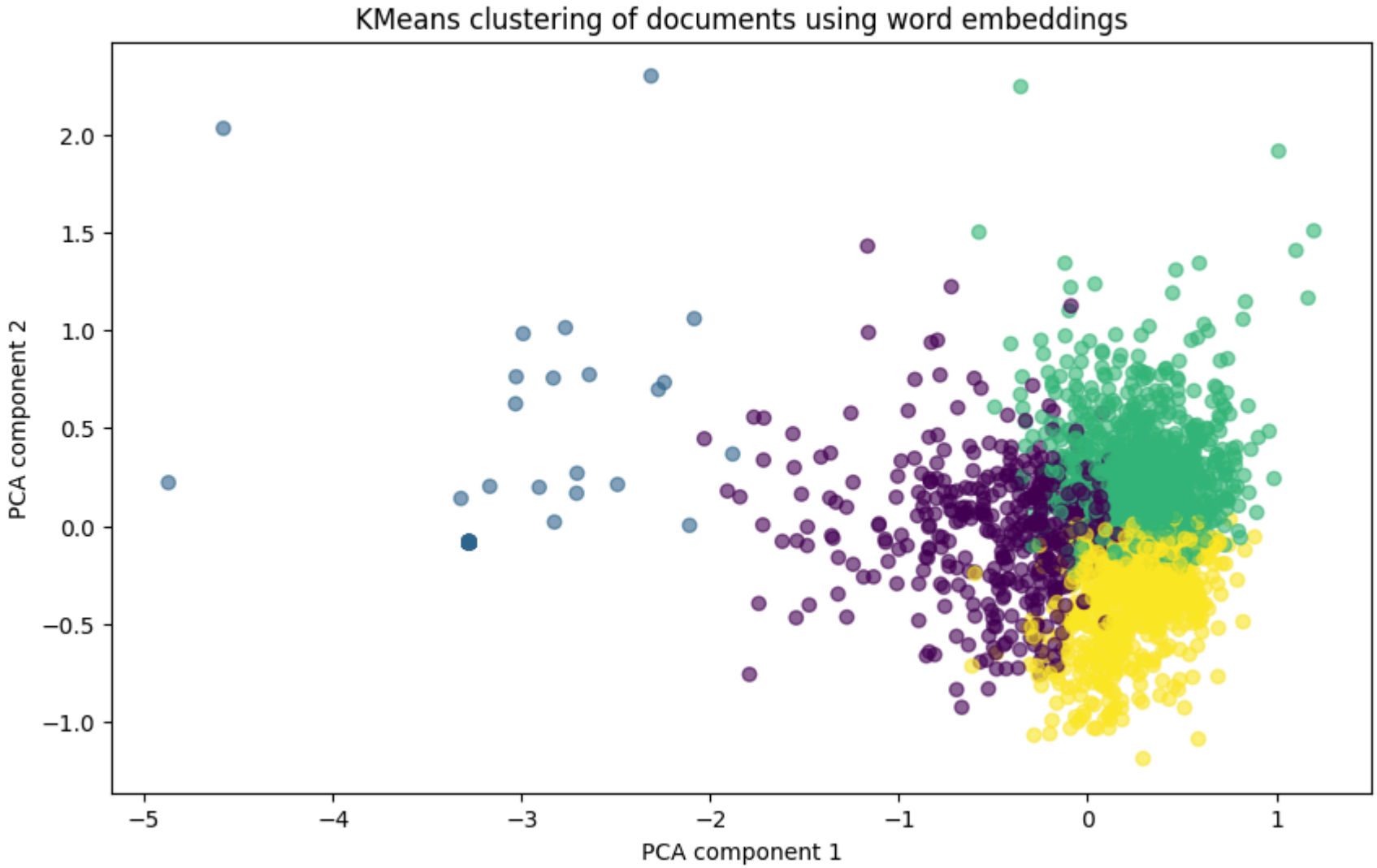
```
# ----------------------------
```

```
# 7. PCA - smanjenje dimenzionalnosti na 2D
# ----------------------------
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)

print("Shape nakon PCA:", X_2d.shape)
```

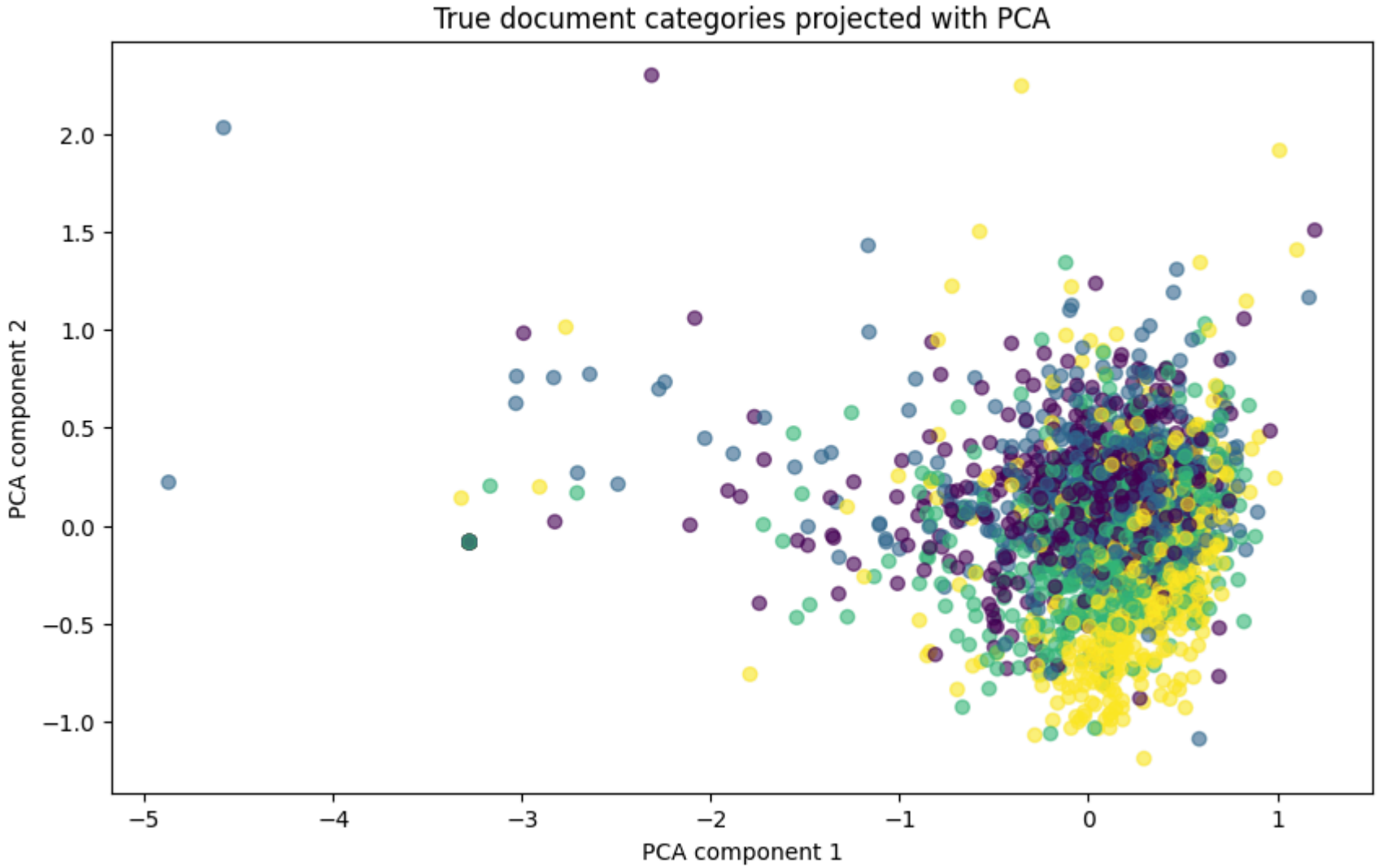
```
Shape nakon PCA: (2338, 2)
```

```
# ----------------------------
# 8. Vizualizacija klastera dobivenih KMeans-om
# ----------------------------
plt.figure(figsize=(10, 6))
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=cluster_labels, alpha=0.6)
plt.title("KMeans clustering of documents using word embeddings")
plt.xlabel("PCA component 1")
plt.ylabel("PCA component 2")
plt.show()
```




```
# ----------------------------
# 9. Vizualizacija stvarnih klasa radi usporedbe
# ----------------------------
plt.figure(figsize=(10, 6))
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=true_labels, alpha=0.6)
plt.title("True document categories projected with PCA")
```

```
plt.xlabel("PCA component 1")
plt.ylabel("PCA component 2")
plt.show()
```



Vježba

Napravite dvije klasifikacije nad istim skupom dokumenata koristeći 20 Newsgroups dataset. Koristite sljedeće kategorije:

sci.med rec.autos comp.windows.x talk.politics.misc

Zadatak je sljedeći:

Prvi put napravite klasifikaciju koristeći TF-IDF i klasifikator po izboru (Naive Bayes ili Logistic Regression). Drugi put napravite klasifikaciju koristeći GloVe embeddinge, gdje svaki dokument predstavljate kao prosjek vektora njegovih riječi, te ponovno primijenite klasifikator.

Na kraju usporedite rezultate i pokušajte objasniti zašto jedna reprezentacija daje bolje ili lošije rezultate.

početni kod za uitavanje dataseta

```
from sklearn.datasets import fetch_20newsgroups

# definiramo kategorije
categories = [
    "sci.med",
    "rec.autos",
    "comp.windows.x",
    "talk.politics.misc"
]

# učitavanje dataseta
dataset = fetch_20newsgroups(
    subset="all",
    categories=categories,
    remove=("headers", "footers", "quotes")
)


texts = dataset.data
labels = dataset.target
target_names = dataset.target_names
```

```
print("Broj dokumenata:", len(texts))
print("Kategorije:", target_names)
```

```
Broj dokumenata: 3743
Kategorije: ['comp.windows.x', 'rec.autos', 'sci.med', 'talk.politics.misc']
```

Primjeri

I sat on the bank of the river. I went to the bank to withdraw money.

Vidio sam svog pasa i zavezao ga remenom oko svog pasa.

Prednosti klasičnih NLP metoda (BoW, TF-IDF, Word2Vec)

- Jednostavne za razumjeti i implementirati
- Brze i računalno jeftine (rade bez GPU-a)
- Dobro rade na manjim datasetovima *Interpretabilne (posebno TF-IDF – vidiš koje riječi nose težinu)
- Dobar baseline za mnoge zadatke (klasifikacija, clustering)
- Lako se kombiniraju s klasičnim ML modelima (Naive Bayes, Logistic Regression)

Nedostaci (zašto nisu dovoljni)

- Ne razumiju kontekst (riječ ima isto značenje svugdje)
- Ignoriraju redoslijed riječi (BoW, TF-IDF)
- Problem s negacijom (“not good” ≠ “good”)
- Loše hvataju sinonime i semantičke odnose (TF-IDF)
- Word2Vec je statičan (ista riječ → isti vektor uvijek)
- Gubi se struktura rečenice i značenje na razini teksta